

Process Calculi & Verified Concurrency

From Dijkstra's Semaphores to Session Types
in Lean 4

A self-contained journey through the theory of
concurrent computation, from labeled transition systems
to verified deadlock-free protocols.

With connections to TypeScript/Node.js
throughout.

Draft — March 16, 2026

Contents

Preface	iv
I Foundations of Communicating Processes	1
1 Why Concurrency Is Hard	2
1.1 The Problem	2
1.2 Concurrency in Node.js	2
1.2.1 The Event Loop	2
1.2.2 Worker Threads	3
1.3 What Can Go Wrong	3
1.3.1 Race Conditions	4
1.3.2 Deadlock	4
1.4 The Way Forward	4
2 Labeled Transition Systems	5
2.1 Processes as State Machines	5
2.2 Traces	6
2.3 Example: A Vending Machine	6
2.4 Determinism and Nondeterminism	6
3 CCS — The Calculus of Communicating Systems	8
3.1 Robin Milner and the Quest for a Theory	8
3.2 CCS Syntax	8
3.3 Operational Semantics	10
3.4 The Communication Rule	10
3.4.1 Worked Example: Request-Response	10
3.4.2 The Node.js Mapping	11
4 Bisimulation — When Are Two Systems “The Same”?	12
4.1 The Coffee Machine Problem	12
4.2 Bisimulation	13
4.3 Bisimulation as a Greatest Fixed Point	13
4.4 Properties of Bisimilarity	14
5 Deadlock, Livelock, and Safety Properties	15
5.1 Dining Philosophers	15
5.2 Deadlock	15
5.2.1 Deadlock in CCS	16
5.3 Livelock	16
5.4 Safety and Liveness	16
5.5 The Dining Philosophers in CCS	17

II	Advanced Theory	18
6	The π-Calculus — Mobile Channels	19
6.1	Beyond Fixed Topologies	19
6.2	π -Calculus Syntax	19
6.3	Communication and Scope Extrusion	20
6.3.1	The Node.js Connection	20
6.4	Encoding the λ -Calculus	21
7	Structural Congruence and Algebraic Laws	22
7.1	The Algebra of Processes	22
7.2	Laws of CCS (Up to Bisimilarity)	22
7.3	Structural Congruence	23
8	Hennessy-Milner Logic	24
8.1	A Logic for Processes	24
8.2	The Adequacy Theorem	25
8.2.1	Distinguishing the Coffee Machines	25
8.3	Specification via Logic	25
III	Types for Concurrency	27
9	Session Types — Protocols as Types	28
9.1	Typing Communication	28
9.2	Session Type Syntax	28
9.3	Duality	29
9.4	Type Safety	30
9.5	The Linear Logic Connection	30
10	Practical Session Types for TypeScript	32
10.1	What Session Types Would Look Like	32
10.2	Current TypeScript Approximations	32
10.2.1	State Machine Pattern	32
10.2.2	Discriminated Unions with XState	33
10.3	What's Missing in TypeScript	33
10.4	Languages with Real Session Type Support	33
11	Multiparty Session Types	35
11.1	Beyond Two Parties	35
11.2	Global Types	35
11.2.1	Example: Client-Server-Database	36
11.3	Projection	36
11.4	Connection to Choreographic Programming	37
IV	Capstone	38
12	Capstone — Verified Worker Pool Protocol	39
12.1	The Protocol	39
12.1.1	Global Type	39
12.1.2	π -Calculus Formalization	39
12.2	Properties to Prove	40

12.3 The TypeScript Implementation	41
13 Looking Back — The Science of Concurrency	44
13.1 The Intellectual Arc	44
13.2 Where the Models Live in Industry	44
13.3 What We Didn't Cover	44
13.4 Cross-Book Connections	45
Appendix A: Quick Reference	46
Appendix B: Lean 4 Definitions Index	47
Appendix C: Historical Timeline	48

Preface

Concurrent programming is everywhere and almost nobody understands it.

Every web server, every mobile app, every database, every operating system relies on concurrent execution. Node.js processes thousands of simultaneous connections through its event loop. Worker threads enable true parallelism in JavaScript. Go’s goroutines and Erlang’s actors build entire distributed systems around message passing. And yet: race conditions, deadlocks, and data corruption remain among the most common and hardest-to-debug classes of software defects. The Therac-25 radiation therapy machine killed patients because of a race condition. The Ariane 5 rocket exploded because of a concurrent sensor processing fault. These are not hypothetical risks.

The root cause is that concurrent programs are *fundamentally different* from sequential ones. When two processes run in parallel, their actions can interleave in exponentially many ways. Testing can explore only a tiny fraction of these interleavings. We need *mathematical proof*.

This book develops the theory of concurrent computation from first principles, formalizes it in Lean 4, and connects it to the TypeScript/Node.js ecosystem you use at work. By the end, you will understand:

- What a *process calculus* is and why it’s the “lambda calculus of concurrency.”
- What *bisimulation* means and why it’s the right notion of equivalence for concurrent systems.
- How to *prove* that a concurrent protocol is deadlock-free.
- What *session types* are and how they can guarantee communication safety at compile time.

Historical arc. The intellectual lineage runs: Dijkstra (1965) identified the problem. Hoare (1978, 1985) and Milner (1980, 1989, 1992) built the theory. Honda (1993) connected concurrency to type theory. Wadler (2012) connected it to linear logic. We follow this arc chronologically.

Prerequisites. Comfort with Lean 4 (inductive types, pattern matching, tactics) and familiarity with TypeScript/Node.js. No prior concurrency theory.

Lean 4 setup. All code is vanilla Lean 4, no Mathlib. We build every structure from scratch.

Part I

Foundations of Communicating Processes

Chapter 1

Why Concurrency Is Hard

“Testing can be used to show the presence of bugs, but never to show their absence.”

Edsger W. Dijkstra

1.1 The Problem

Dijkstra and the Birth of Concurrency Theory

In 1965, Edsger Dijkstra published “Cooperating Sequential Processes,” a technical report that identified the fundamental problem of concurrent programming: when multiple processes share resources, things go wrong in ways that are almost impossible to detect by testing. A sequential program always does the same thing given the same input. A concurrent program can do *different* things on different runs, depending on the precise timing of events—the *interleaving* of actions.

Dijkstra introduced the **mutual exclusion problem**: given n processes that need occasional access to a shared resource, design a protocol ensuring that at most one process uses the resource at any time. His solution used **semaphores**—integer variables with atomic increment and decrement operations—the first general-purpose synchronization primitive. His 1968 paper “Go To Statement Considered Harmful” made him famous for structured programming, but his concurrency work is arguably his deepest contribution. Dijkstra received the Turing Award in 1972. His acceptance lecture contained the memorable line: “The question of whether machines can think is about as relevant as the question of whether submarines can swim.”

1.2 Concurrency in Node.js

As a TypeScript/Node.js programmer, you deal with concurrency every day—even if you don’t always call it that.

1.2.1 The Event Loop

Node.js is single-threaded but *asynchronous*. The event loop processes callbacks one at a time, but I/O operations run in the background:

TypeScript

```
// These run "concurrently" --- the event loop interleaves them
const fetchUser = async (id: string) => {
  const resp = await fetch(`/api/users/${id}`); // yields to event loop
  return resp.json();
};

// Two concurrent fetches
const [alice, bob] = await Promise.all([
  fetchUser("alice"),
  fetchUser("bob"),
]);
```

This is *cooperative concurrency*: each task voluntarily yields at `await` points. No two callbacks run simultaneously, so you avoid many classical race conditions. But not all of them—the interleaving of callbacks can still surprise you.

1.2.2 Worker Threads

For CPU-intensive work, Node.js provides `Worker` threads—true parallelism:

TypeScript

```
import { Worker, parentPort, MessageChannel } from 'worker_threads';

// Main thread
const worker = new Worker('./worker.js');
worker.postMessage({ task: 'compute', data: payload });
worker.on('message', (result) => {
  console.log('Got result:', result);
});

// Worker thread (worker.js)
parentPort?.on('message', (msg) => {
  const result = heavyComputation(msg.data);
  parentPort?.postMessage(result);
});
```

The moment you use workers, you enter Dijkstra’s world: two threads running *truly simultaneously*, communicating via message passing. The `postMessage/on('message')` pattern is exactly what process calculi formalize.

1.3 What Can Go Wrong

1.3.1 Race Conditions

TypeScript

```
// Shared state between event loop callbacks
let balance = 100;

async function withdraw(amount: number) {
  if (balance >= amount) {           // Check
    await delay(10);                 // Simulate async work
    balance -= amount;               // Act --- but balance may have changed!
  }
}

// Two concurrent withdrawals: both see balance = 100
await Promise.all([withdraw(80), withdraw(80)]);
// balance is now -60, not 20!
```

1.3.2 Deadlock

TypeScript

```
// Worker A waits for B's response before sending its own
// Worker B waits for A's response before sending its own
// Neither can proceed: DEADLOCK

workerA.postMessage('waiting for B'); // A blocks until B responds
workerB.postMessage('waiting for A'); // B blocks until A responds
```

The Therac-25

The Therac-25 (1985–1987) was a radiation therapy machine whose software had a race condition between the operator interface and the beam control system. Under specific timing—an operator typing faster than the system expected—the machine delivered radiation doses hundreds of times higher than intended. Six patients were seriously injured; three died. The bug was virtually impossible to reproduce in testing because it depended on exact keystroke timing. This is the canonical example of why concurrent systems need mathematical reasoning, not just testing.

1.4 The Way Forward

The rest of this book develops mathematical tools for reasoning about concurrent systems with *certainty*:

- **Labeled transition systems** (Chapter 2): model concurrent systems as state machines.
 - **CCS** (Chapter 3): a “programming language” for concurrent processes.
 - **Bisimulation** (Chapter 4): when are two concurrent systems equivalent?
 - **Session types** (Chapter 9): type systems that guarantee deadlock freedom.
- Every concept will be formalized in Lean 4 and connected back to TypeScript.

Chapter 2

Labeled Transition Systems

“The nice thing about standards is that you have so many to choose from.”

Andrew S. Tanenbaum

2.1 Processes as State Machines

Keller and the LTS

Robert Keller (1976) introduced labeled transition systems as a mathematical semantics for concurrent programs. The idea: a process is a state machine whose transitions are labeled with *actions*—things the process does that the outside world can observe—or with a special silent action τ (something internal the observer cannot see). This framework became the standard foundation for all subsequent process calculi.

Definition 2.1 — Labeled Transition System

A **labeled transition system (LTS)** is a triple $(S, A, \rightarrow)$ where:

- S is a set of **states**.
- A is a set of **actions** (labels), including the **silent action** τ .
- $\rightarrow \subseteq S \times A \times S$ is the **transition relation**. We write $s \xrightarrow{a} s'$ for $(s, a, s') \in \rightarrow$.

Lean 4

```
-- Actions: either observable or silent (tau)
inductive Action (Label : Type) where
| obs : Label → Action Label -- observable action
| tau : Action Label         -- internal/silent action
deriving DecidableEq, Repr

-- A labeled transition system
structure LTS (State Label : Type) where
  step : State → Action Label → State → Prop
```

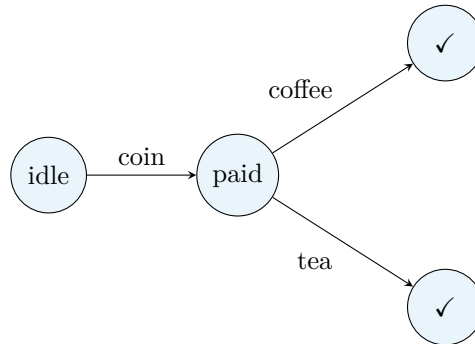
2.2 Traces

A **trace** of a process is a sequence of *observable* actions it can perform (ignoring τ steps). Formally, the trace set of state s is:

$$\text{traces}(s) = \{a_1 a_2 \cdots a_n : \exists s_0, \dots, s_n, s = s_0 \Longrightarrow^* \xrightarrow{a_1} \Longrightarrow^* \xrightarrow{a_2} \cdots \xrightarrow{a_n} s_n\}$$

where $\Longrightarrow^*$ denotes zero or more τ steps.

2.3 Example: A Vending Machine



This machine accepts a coin, then dispenses either coffee or tea. Its trace set is $\{\text{coin} \cdot \text{coffee}, \text{coin} \cdot \text{tea}\}$.

Lean 4

```

-- Vending machine states
inductive VMState where
  | idle | paid | done
  deriving DecidableEq, Repr

-- Vending machine actions
inductive VMLabel where
  | coin | coffee | tea
  deriving DecidableEq, Repr

-- Transition relation
inductive VMStep : VMState → Action VMLabel → VMState → Prop where
  | insert : VMStep .idle (.obs .coin) .paid
  | getCoffee : VMStep .paid (.obs .coffee) .done
  | getTea : VMStep .paid (.obs .tea) .done

```

2.4 Determinism and Nondeterminism

The vending machine is **nondeterministic**: from state **paid**, there are two possible transitions (coffee or tea). The “choice” is made by the environment (the customer), not the machine.

Definition 2.2 — Deterministic LTS

An LTS is **deterministic** if for every state s and action a , there is at most one state s' with $s \xrightarrow{a} s'$.

Most interesting concurrent systems are nondeterministic: the interleaving of parallel processes creates multiple possible execution paths.

Exercise 2.1

Model a traffic light (green $\rightarrow$ yellow $\rightarrow$ red $\rightarrow$ green) as an LTS in Lean. Prove it is deterministic.

Exercise 2.2

Model a retry-with-backoff mechanism: a process that sends a request, either receives a response (success) or times out (failure), and on failure retries up to 3 times. Define the LTS and compute its trace set.

Chapter 3

CCS — The Calculus of Communicating Systems

“There is no concurrency theory without Robin Milner.”

Luca Cardelli

3.1 Robin Milner and the Quest for a Theory

Robin Milner (1934–2010)

Robin Milner is one of the most important computer scientists in history. He invented three things, each of which alone would be a career-defining achievement:

ML (1973): the programming language family that includes OCaml and F#, and whose type inference algorithm (Algorithm W) inspired Haskell and Lean’s type systems. Milner proved that “well-typed programs do not go wrong”—the first type soundness theorem.

CCS (1980): the Calculus of Communicating Systems, a minimal formal language for concurrent processes. Just as Church’s lambda calculus captures sequential computation with three constructs (variable, abstraction, application), CCS captures concurrent computation with a small set of operators (inaction, prefix, choice, parallel, restriction).

The π -calculus (1992, with Parrow and Walker): an extension of CCS where channel names can be communicated, modeling systems whose communication topology changes at runtime.

Milner received the Turing Award in 1991 “for three distinct and complete achievements”—an unprecedented citation. His 1989 book *Communication and Concurrency* remains the definitive introduction to the theory.

3.2 CCS Syntax

CCS processes are built from six constructors:

Definition 3.1 — CCS Syntax

Given a set of **names** (channels) $\mathcal{N}$, the set of CCS processes is defined by:

$$P, Q ::= 0 \mid \alpha . P \mid P + Q \mid P \parallel Q \mid (\nu a)P \mid !P$$

where α is an **action**: either a name a (input), a co-name $\bar{a}$ (output), or τ (silent).

What each constructor means:

- 0 — **inaction**. The stopped process; does nothing.
- $\alpha.P$ — **prefix**. Perform action α , then continue as P .
- $P + Q$ — **choice**. Behave as P or Q ; the environment decides which.
- $P \parallel Q$ — **parallel composition**. P and Q run concurrently.
- $(\nu a)P$ — **restriction**. Channel a is private to P ; the outside world cannot use it.
- $!P$ — **replication**. Unbounded copies of P ; models a server that can handle any number of requests.

Lean 4

```
-- Channel names
abbrev Name := String

-- CCS actions
inductive CCSAct where
  | inp  : Name → CCSAct      -- input on channel a
  | out  : Name → CCSAct      -- output on channel ā
  | tau  : CCSAct             -- silent action
  deriving DecidableEq, Repr

-- CCS is the complement of an action
def CCSAct.complement : CCSAct → Option CCSAct
  | .inp a => some (.out a)
  | .out a => some (.inp a)
  | .tau   => none

-- CCS processes
inductive CCS where
  | nil  : CCS                -- 0
  | pre  : CCSAct → CCS → CCS -- α.P
  | plus : CCS → CCS → CCS    -- P + Q
  | par  : CCS → CCS → CCS    -- P | Q
  | res  : Name → CCS → CCS    -- (νa)P
  | repl : CCS → CCS          -- !P
  deriving Repr

-- Notation helpers
scoped notation "□" => CCS.nil
scoped infixr:60 " □ " => CCS.pre      -- α.P
scoped infixl:50 " ⊕ " => CCS.plus      -- P + Q
scoped infixl:40 " □ " => CCS.par       -- P | Q
```

3.3 Operational Semantics

The *meaning* of a CCS process is given by its operational semantics: an inductive relation defining which transitions are possible.

Definition 3.2 — CCS Operational Semantics

The transition relation $P \xrightarrow{\alpha} P'$ is the smallest relation satisfying these rules:

- (a) Prefix: $\alpha . P \xrightarrow{\alpha} P$
- (b) Choice-L: If $P \xrightarrow{\alpha} P'$ then $P + Q \xrightarrow{\alpha} P'$
- (c) Choice-R: If $Q \xrightarrow{\alpha} Q'$ then $P + Q \xrightarrow{\alpha} Q'$
- (d) Par-L: If $P \xrightarrow{\alpha} P'$ then $P \parallel Q \xrightarrow{\alpha} P' \parallel Q$
- (e) Par-R: If $Q \xrightarrow{\alpha} Q'$ then $P \parallel Q \xrightarrow{\alpha} P \parallel Q'$
- (f) Comm: If $P \xrightarrow{a} P'$ and $Q \xrightarrow{\bar{a}} Q'$ then $P \parallel Q \xrightarrow{\tau} P' \parallel Q'$
- (g) Res: If $P \xrightarrow{\alpha} P'$ and $\alpha \neq a, \bar{a}$ then $(\nu a)P \xrightarrow{\alpha} (\nu a)P'$
- (h) Repl: If $P \parallel !P \xrightarrow{\alpha} P'$ then $!P \xrightarrow{\alpha} P'$

Lean 4

```
-- CCS transition relation
inductive CCSStep : CCS → CCSAct → CCS → Prop where
| prefix : CCSStep (.pre α P) α P
| choiceL : CCSStep P α P' → CCSStep (.plus P Q) α P'
| choiceR : CCSStep Q α Q' → CCSStep (.plus P Q) α Q'
| parL : CCSStep P α P' → CCSStep (.par P Q) α (.par P' Q)
| parR : CCSStep Q α Q' → CCSStep (.par P Q) α (.par P Q')
| comm : CCSStep P (.inp a) P' → CCSStep Q (.out a) Q' →
  CCSStep (.par P Q) .tau (.par P' Q')
| commR : CCSStep P (.out a) P' → CCSStep Q (.inp a) Q' →
  CCSStep (.par P Q) .tau (.par P' Q')
| res : CCSStep P α P' → α ≠ .inp a → α ≠ .out a →
  CCSStep (.res a P) α (.res a P')
```

3.4 The Communication Rule

The most important rule is Comm: when two parallel processes perform complementary actions (a and $\bar{a}$) on the same channel, they **synchronize** via a silent τ step. This models message passing: one process sends, the other receives, and the communication is invisible to the outside.

3.4.1 Worked Example: Request-Response

$$\text{Client} = \overline{\text{req}} . \text{resp} . \mathbf{0}$$

$$\text{Server} = \text{req} . \overline{\text{resp}} . \mathbf{0}$$

$$\begin{aligned} \text{Client} \parallel \text{Server} &\xrightarrow{\tau} \text{resp} . \mathbf{0} \parallel \overline{\text{resp}} . \mathbf{0} && (\text{Comm: req}/\overline{\text{req}}) \\ &\xrightarrow{\tau} \mathbf{0} \parallel \mathbf{0} && (\text{Comm: resp}/\overline{\text{resp}}) \\ &= \mathbf{0} \parallel \mathbf{0} && (\text{done}) \end{aligned}$$

Two τ steps: two internal communications. The outside world sees nothing —the entire exchange is private.

3.4.2 The Node.js Mapping

CCS		Node.js
$P \parallel Q$	$\longleftrightarrow$	two <code>Worker</code> threads
$a / \bar{a}$	$\longleftrightarrow$	<code>port.on('message') / port.postMessage()</code>
$(\nu a)P$	$\longleftrightarrow$	<code>new MessageChannel()</code> local to a module
$P + Q$	$\longleftrightarrow$	<code>Promise.race([p, q])</code>
$!P$	$\longleftrightarrow$	server listening in a loop

Exercise 3.1

Model a pub/sub system in CCS: a publisher sends messages on a channel, and two subscribers receive them. Define the processes and trace through the possible executions.

Exercise 3.2

Prove in Lean that the Client $\parallel$ Server system above reaches $\mathbf{0} \parallel \mathbf{0}$ after exactly two τ steps, by constructing the derivation using `CCSStep` constructors.

Chapter 4

Bisimulation — When Are Two Systems “The Same”?

“Two systems are equivalent if no experiment can distinguish them.”

Robin Milner (paraphrased)

4.1 The Coffee Machine Problem

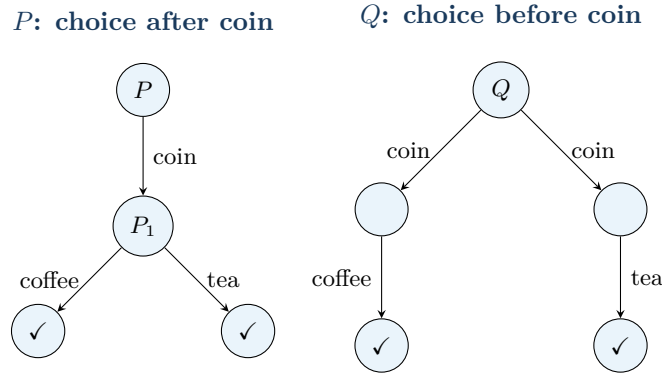
Park and Milner

After defining CCS, Milner faced a subtle question: when should two processes be considered equivalent? The naive answer—“they can perform the same sequences of actions” (trace equivalence)—turns out to be too coarse. David Park (1981) and Milner independently arrived at the right notion: **bisimulation**. Two processes are bisimilar if every step one can take, the other can match—step by step, preserving the branching structure, not just the complete traces.

The canonical counterexample:

$$\begin{aligned}P &= \text{coin} . (\text{coffee} . \mathbf{0} + \text{tea} . \mathbf{0}) \\Q &= \text{coin} . \text{coffee} . \mathbf{0} + \text{coin} . \text{tea} . \mathbf{0}\end{aligned}$$

Both have the same traces: $\{\text{coin} \cdot \text{coffee}, \text{coin} \cdot \text{tea}\}$. But they behave differently! In P , after inserting a coin you can still choose coffee or tea. In Q , the choice was already made (by the nondeterministic resolution of $+$) *before* you inserted the coin. An observer who inserts a coin and then tries to get coffee might fail with Q (if the second branch was chosen) but never with P .



4.2 Bisimulation

Definition 4.1 — Bisimulation

Let $(S, A, \rightarrow)$ be an LTS. A binary relation $R \subseteq S \times S$ is a **bisimulation** if whenever $s R t$:

- (i) If $s \xrightarrow{a} s'$, then there exists t' such that $t \xrightarrow{a} t'$ and $s' R t'$.
- (ii) If $t \xrightarrow{a} t'$, then there exists s' such that $s \xrightarrow{a} s'$ and $s' R t'$.

Two states s, t are **bisimilar** (written $s \sim t$) if there exists a bisimulation R with $s R t$.

Lean 4

```

-- A relation R is a bisimulation
def IsBisimulation {State Label : Type}
  (step : State → Action Label → State → Prop)
  (R : State → State → Prop) : Prop :=
  ∀ s t, R s t →
    (∀ a s', step s a s' → ∃ t', step t a t' ∧ R s' t') ∧
    (∀ a t', step t a t' → ∃ s', step s a s' ∧ R s' t')

-- Bisimilarity: exists a bisimulation relating them
def Bisimilar {State Label : Type}
  (step : State → Action Label → State → Prop)
  (s t : State) : Prop :=
  ∃ R, IsBisimulation step R ∧ R s t
  
```

4.3 Bisimulation as a Greatest Fixed Point

Intuition

Bisimilarity has a beautiful lattice-theoretic characterization. Define a function F on relations: $F(R) = \{(s, t) \mid R \text{ satisfies the bisimulation conditions at } (s, t)\}$. Then bisimilarity is the **greatest fixed point** of F —the largest relation that is a bisimulation.

This is the Knaster-Tarski theorem from lattice theory! The lattice of relations on S (ordered by inclusion) is complete, F is monotone, and its greatest fixed point exists and equals the union of all bisimulations.

If you’ve studied the lattice/propagator book, this connection is direct: bisimilarity is computed by the same fixed-point machinery that computes propagator convergence.

4.4 Properties of Bisimilarity

Theorem 4.1 — Bisimilarity is an Equivalence Relation

$\sim$ is reflexive, symmetric, and transitive.

Proof. Reflexive: the identity relation is a bisimulation. Symmetric: if R is a bisimulation, so is R^{-1} . Transitive: if R_1 and R_2 are bisimulations, then $R_1 \circ R_2$ is a bisimulation (this requires showing that the composed relation satisfies both conditions). $\square$

Theorem 4.2 — Bisimilarity is a Congruence

If $P \sim Q$, then: $\alpha . P \sim \alpha . Q$, $P + R \sim Q + R$, $P \parallel R \sim Q \parallel R$, $(\nu a)P \sim (\nu a)Q$.

This means bisimilar processes can be substituted for each other in any context—the analogue of the “substitution principle” in programming.

Exercise 4.1

Prove that $P \parallel Q \sim Q \parallel P$ (parallel composition is commutative up to bisimulation). Construct the bisimulation relation explicitly.

Exercise 4.2

Prove that P and Q (the coffee machines above) are *not* bisimilar. *Hint:* after the coin step, P reaches a state that can do both coffee and tea, but Q reaches a state that can do only one.

Chapter 5

Deadlock, Livelock, and Safety Properties

“Simplicity is a great virtue but it requires hard work to achieve it and education to appreciate it.”

Edsger W. Dijkstra

5.1 Dining Philosophers

Dijkstra’s Dining Philosophers (1971)

Five philosophers sit around a circular table. Between each pair is a single fork. To eat, a philosopher needs both adjacent forks. Each philosopher alternates between thinking and eating. If every philosopher simultaneously picks up their left fork, then all wait forever for the right fork: **deadlock**.

Dijkstra introduced this problem to illustrate that deadlock is not a bug in any individual component—each philosopher follows a perfectly reasonable protocol. Deadlock is a property of the *interaction*, arising from circular dependencies.

5.2 Deadlock

Definition 5.1 — Deadlock

A CCS process P is **deadlocked** if:

- (i) $P \neq \mathbf{0}$ (the process hasn’t terminated).
- (ii) There is no α and P' such that $P \xrightarrow{\alpha} P'$ (no transitions possible).

Lean 4

```
def isDeadlocked (P : CCS) : Prop :=  
  P ≠ .nil ∧ ∀ α P', ¬ CCSStep P α P'
```

5.2.1 Deadlock in CCS

$$\begin{aligned}
 A &= \bar{b}.a.0 && (\text{sends on } b, \text{ then waits on } a) \\
 B &= \bar{a}.b.0 && (\text{sends on } a, \text{ then waits on } b) \\
 &(\nu a)(\nu b)(A \parallel B)
 \end{aligned}$$

Both A and B want to send first. Under restriction, neither output is visible externally, and neither can synchronize because A outputs on b while B outputs on a —but A is waiting for input on a (which B hasn't sent yet) and B is waiting for input on b (which A hasn't sent yet). Deadlock.

5.3 Livelock

Definition 5.2 — Livelock / Divergence

A process P **diverges** (livelocks) if it can perform an infinite sequence of τ steps without ever performing an observable action.

In TypeScript terms: a `setImmediate` loop that prevents the event loop from processing any other callbacks.

5.4 Safety and Liveness

Definition 5.3 — Safety Property

A **safety property** asserts: “nothing bad ever happens.” Formally, a predicate φ holds in all reachable states.

Definition 5.4 — Liveness Property

A **liveness property** asserts: “something good eventually happens.” Formally, every execution eventually reaches a state satisfying φ .

Hoare and CSP

Tony Hoare's *Communicating Sequential Processes* (1978 paper, 1985 book) is the other major process calculus, developed independently from Milner's CCS. Hoare focused on *traces and refinement*—specifying what a system should do and verifying that an implementation refines the specification. CSP directly influenced Go's goroutines and channels; Rob Pike, one of Go's creators, cites Hoare as the primary inspiration. CSP and CCS are closely related but have different philosophies: CSP focuses on traces (good for specification), CCS on bisimulation (good for equivalence). Both are Turing-complete and can encode each other.

5.5 The Dining Philosophers in CCS

Lean 4

```

-- Philosopher i picks up fork i, then fork (i+1) mod 5, eats, puts both down
def philosopher (i : Fin 5) : CCS :=
  let li := s!"pick{i}"
  let ri := s!"pick{(i.val + 1) % 5}"
  let di := s!"drop{i}"
  let dri := s!"drop{(i.val + 1) % 5}"
  .pre (.inp li) <|      -- pick up left fork
  .pre (.inp ri) <|      -- pick up right fork
  .pre (.out di) <|      -- put down left fork
  .pre (.out dri) <|     -- put down right fork
  .nil

-- The asymmetric fix: philosopher 4 picks up forks in reverse order
def philosopher4_fixed : CCS :=
  .pre (.inp "pick0") <|  -- right fork first!
  .pre (.inp "pick4") <|  -- then left fork
  .pre (.out "drop0") <|
  .pre (.out "drop4") <|
  .nil

```

The asymmetric solution breaks the circular dependency: philosopher 4 picks up fork 0 first (instead of fork 4), which means at least one philosopher can always make progress.

Exercise 5.1

Model a reader-writer lock in CCS: multiple readers can hold the lock simultaneously, but a writer needs exclusive access. Prove (informally, or in Lean) that the protocol is deadlock-free.

Exercise 5.2

Formalize the deadlock property in Lean for the symmetric dining philosophers (all pick up left fork first). Show that a deadlocked configuration exists.

Part II

Advanced Theory

Chapter 6

The π -Calculus — Mobile Channels

“Functions are processes, and processes are functions.”

Robin Milner

6.1 Beyond Fixed Topologies

Milner, Parrow, and Walker (1992)

CCS has a limitation: the set of channel names is fixed at design time. In real distributed systems, communication topologies change at runtime: a server hands off a client connection to a worker; a chat participant invites a new user; a microservice discovers another via a registry.

In 1992, Milner (with Joachim Parrow and David Walker) introduced the π -calculus, extending CCS with one radical new feature: **channel names can be communicated over channels**. This seems like a small change— $\bar{x}\langle y \rangle.P$ means “send the name y over channel x , then continue as P ”—but its consequences are profound. It models any system whose communication graph evolves dynamically.

Milner’s tour de force was showing that the λ -calculus can be *encoded* in the π -calculus: function application becomes communication, closures become restricted channels. This means concurrent computation is at least as fundamental as sequential computation.

6.2 π -Calculus Syntax

Definition 6.1 — π -Calculus

$$P, Q ::= \mathbf{0} \mid \bar{x}\langle y \rangle.P \mid x(y).P \mid P \parallel Q \mid (\nu a)P \mid !P \mid P + Q$$

- $\bar{x}\langle y \rangle.P$ — send name y over channel x .
- $x(y).P$ — receive a name on channel x , bind it to y , continue as P with y replaced by whatever was received.

Lean 4

```

--  $\pi$ -calculus processes
inductive Pi where
| nil  : Pi
| send : Name → Name → Pi → Pi      --  $\bar{x}(y).P$ 
| recv : Name → Name → Pi → Pi      --  $x(y).P$  ( $y$  is bound)
| par  : Pi → Pi → Pi                --  $P \mid Q$ 
| res  : Name → Pi → Pi              --  $(\nu a)P$ 
| repl : Pi → Pi                    --  $!P$ 
| plus : Pi → Pi → Pi                --  $P + Q$ 
deriving Repr

-- Free names
def Pi.freeNames : Pi → List Name
| .nil => []
| .send x y P => [x, y] ++ P.freeNames
| .recv x y P => [x] ++ (P.freeNames.filter (· ≠ y)) --  $y$  is bound
| .par P Q => P.freeNames ++ Q.freeNames
| .res a P => P.freeNames.filter (· ≠ a)
| .repl P => P.freeNames
| .plus P Q => P.freeNames ++ Q.freeNames

```

6.3 Communication and Scope Extrusion

The communication rule: when $\bar{x}(y).P$ runs in parallel with $x(z).Q$, they synchronize— y is sent, z gets bound to y :

$$\bar{x}(y).P \parallel x(z).Q \xrightarrow{\tau} P \parallel Q[y/z]$$

The truly novel feature is **scope extrusion**: if a private name escapes via communication, its scope automatically widens.

$$(\nu a)(\bar{x}(a).P) \parallel x(z).Q \xrightarrow{\tau} (\nu a)(P \parallel Q[a/z])$$

The name a was private to the left process, but after sending it over x , the right process now knows a too. The νa scope expands to cover both processes.

6.3.1 The Node.js Connection

This is *exactly* what happens with `MessageChannel` and transferable objects:

TypeScript

```

// Main thread
const { port1, port2 } = new MessageChannel(); // (vport1)(vport2)

// Send port2 to the worker --- scope extrusion!
worker.postMessage({ port: port2 }, [port2]);

// Now the worker has port2 and can communicate on it directly
// The "private" channel has escaped its original scope

```

6.4 Encoding the λ -Calculus

Milner showed that function application can be encoded as communication:

$$\begin{aligned}
\llbracket x \rrbracket_u &= \bar{x}\langle u \rangle. \mathbf{0} \\
\llbracket \lambda x. M \rrbracket_u &= u((x, v)). \llbracket M \rrbracket_v \\
\llbracket M N \rrbracket_u &= (\nu v)(\llbracket M \rrbracket_v \parallel v((x, w)).(\llbracket N \rrbracket_x \parallel \bar{w}\langle u \rangle. \mathbf{0}))
\end{aligned}$$

The variable u is a “return channel”—where the result will be sent. Function application creates a fresh channel v , evaluates the function, receives its body, evaluates the argument, and forwards the result.

Intuition

This encoding reveals something deep: sequential computation (the λ -calculus) is a special case of concurrent computation (the π -calculus) where everything is forced to happen in a specific order by the channel dependencies. Concurrency is the more fundamental notion; sequentiality is concurrency with extra constraints.

Exercise 6.1

Model a chat room in the π -calculus: a room process that accepts join requests, and when two users have joined, creates a private channel between them. Use scope extrusion to pass the private channel to both users.

Exercise 6.2

Encode the λ -term $(\lambda x. x) y$ (identity applied to y) using Milner’s encoding. Trace the reduction steps and verify that the result is y .

Chapter 7

Structural Congruence and Algebraic Laws

“Algebra is the offer made by the devil to the mathematician.”

Michael Atiyah

7.1 The Algebra of Processes

Process Algebra

Just as high-school algebra studies equations between numerical expressions ($a + b = b + a$, $a \cdot 1 = a$), **process algebra** studies equations between concurrent processes. The discovery that processes obey clean algebraic laws—and that these laws characterize bisimilarity—was one of the early surprises of concurrency theory. The term “process algebra” was coined by Jan Bergstra and Jan Willem Klop in the early 1980s; their Algebra of Communicating Processes (ACP) developed this algebraic perspective alongside CCS and CSP.

7.2 Laws of CCS (Up to Bisimilarity)

Theorem 7.1 — Algebraic Laws

The following hold for all CCS processes P, Q, R :

$$P \parallel \mathbf{0} \sim P \quad (\text{parallel identity}) \quad (7.1)$$

$$P \parallel Q \sim Q \parallel P \quad (\text{parallel commutativity}) \quad (7.2)$$

$$(P \parallel Q) \parallel R \sim P \parallel (Q \parallel R) \quad (\text{parallel associativity}) \quad (7.3)$$

$$P + \mathbf{0} \sim P \quad (\text{choice identity}) \quad (7.4)$$

$$P + Q \sim Q + P \quad (\text{choice commutativity}) \quad (7.5)$$

$$P + P \sim P \quad (\text{choice idempotence}) \quad (7.6)$$

Intuition

Parallel composition ($\parallel, \mathbf{0}$) forms a **commutative monoid**—the same algebraic structure that addition on integers forms! If you’ve studied the Galois theory book, you recognize this: groups, rings, and fields all have monoid substructure. The algebra of processes and the algebra of numbers share the same abstract patterns.

7.3 Structural Congruence

Definition 7.1 — Structural Congruence

Structural congruence ($\equiv$) is the smallest equivalence relation on CCS/ π -calculus processes that includes the algebraic laws above plus:

$$(\nu a)\mathbf{0} \equiv \mathbf{0} \quad \text{(vacuous restriction)} \quad (7.7)$$

$$(\nu a)(\nu b)P \equiv (\nu b)(\nu a)P \quad \text{(restriction commutes)} \quad (7.8)$$

$$(\nu a)(P \parallel Q) \equiv P \parallel (\nu a)Q \quad \text{if } a \notin \text{fn}(P) \quad \text{(scope extrusion)} \quad (7.9)$$

$$!P \equiv P \parallel !P \quad \text{(replication unfolds)} \quad (7.10)$$

Lean 4

```
-- Structural congruence as an inductive relation
inductive StructCong : CCS → CCS → Prop where
| parComm  : StructCong (.par P Q) (.par Q P)
| parAssoc : StructCong (.par (.par P Q) R) (.par P (.par Q R))
| parNil   : StructCong (.par P .nil) P
| plusComm : StructCong (.plus P Q) (.plus Q P)
| plusNil  : StructCong (.plus P .nil) P
| plusIdem : StructCong (.plus P P) P
| refl     : StructCong P P
| symm     : StructCong P Q → StructCong Q P
| trans    : StructCong P Q → StructCong Q R → StructCong P R
-- congruence: structural congruence is preserved by all operators
| parCong  : StructCong P P' → StructCong (.par P Q) (.par P' Q)
| plusCong : StructCong P P' → StructCong (.plus P Q) (.plus P' Q)
```

Structural congruence is used to “prepare” terms before reduction. The idea: first rearrange using $\equiv$ (which is purely syntactic), then apply the communication rule.

Exercise 7.1

Prove in Lean that `StructCong` is an equivalence relation (reflexive, symmetric, transitive—these are built into the definition above, but verify that the constructors suffice).

Chapter 8

Hennessy-Milner Logic

“Logic is the hygiene of the mathematician.”

André Weil

8.1 A Logic for Processes

Hennessy and Milner (1985)

Matthew Hennessy and Robin Milner introduced a modal logic specifically for reasoning about processes. The logic has two modalities: $\langle a \rangle \varphi$ (“the process *can* do action a and then satisfy φ ”) and $[a] \varphi$ (“*whenever* the process does a , the result satisfies φ ”). The landmark result: two processes are bisimilar if and only if they satisfy exactly the same HM formulas. This means bisimulation is not just one definition of equivalence—it is *the* equivalence that respects all observable properties.

Definition 8.1 — Hennessy-Milner Logic

Given an action set A , the formulas of HM logic are:

$$\varphi, \psi ::= \text{tt} \mid \text{ff} \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \langle a \rangle \varphi \mid [a] \varphi$$

- $\langle a \rangle \varphi$ (diamond): “can do a and then satisfy φ .”
- $[a] \varphi$ (box): “every a -step leads to φ .”

Lean 4

```

-- HM Logic formulas
inductive HML (Label : Type) where
| tt   : HML Label
| ff   : HML Label
| conj : HML Label → HML Label → HML Label
| disj : HML Label → HML Label → HML Label
| dia  : Action Label → HML Label → HML Label -- ⟨a⟩φ
| box  : Action Label → HML Label → HML Label -- [a]φ
deriving Repr

-- Satisfaction: when does a state satisfy a formula?
def Satisfies {State Label : Type}
  (step : State → Action Label → State → Prop)
  : State → HML Label → Prop
| _, .tt => True
| _, .ff => False
| s, .conj φ ψ => Satisfies step s φ ∧ Satisfies step s ψ
| s, .disj φ ψ => Satisfies step s φ ∨ Satisfies step s ψ
| s, .dia a φ  => ∃ s', step s a s' ∧ Satisfies step s' φ
| s, .box a φ  => ∀ s', step s a s' → Satisfies step s' φ

```

8.2 The Adequacy Theorem

Theorem 8.1 — HM Logic Characterizes Bisimilarity

For image-finite LTSs (each state has finitely many transitions):

$$s \sim t \iff \forall \varphi \in \text{HML}, s \models \varphi \iff t \models \varphi$$

The $\Leftarrow$ direction is proved by showing that “satisfies the same formulas” is itself a bisimulation. The $\Rightarrow$ direction is proved by induction on the formula structure.

8.2.1 Distinguishing the Coffee Machines

Remember P (choice after coin) and Q (choice before coin) from Chapter 4? The formula $\langle \text{coin} \rangle (\langle \text{coffee} \rangle \text{tt} \wedge \langle \text{tea} \rangle \text{tt})$ distinguishes them:

- $P \models$ the formula: after coin, both coffee and tea are possible.
- $Q \not\models$ the formula: after coin (choosing the coffee branch), tea is not possible.

8.3 Specification via Logic

HM logic gives us a specification language for concurrent systems:

- “The server can always accept a request”: $[\tau]^* \langle \text{req} \rangle \text{tt}$ (after any internal steps, a request action is possible).
- “After login, logout is always possible”: $[\text{login}] \langle \text{logout} \rangle \text{tt}$.
- “The system never reaches a deadlocked state”: $[a_1] \text{live} \wedge [a_2] \text{live} \wedge \dots$ where $\text{live} = \bigvee_a \langle a \rangle \text{tt}$.

From HM Logic to Model Checking

HM logic is the ancestor of the temporal logics (CTL, CTL*, LTL) used by industrial model-checking tools like SPIN, NuSMV, and TLA+. Edmund Clarke, Allen Emerson, and Joseph Sifakis received the 2007 Turing Award for model checking—the automatic verification of temporal logic properties against finite-state models. Amazon uses TLA+ to verify protocols in DynamoDB, S3, and other AWS services.

Exercise 8.1

Write HM formulas expressing: (a) “the process can terminate” and (b) “the process is deadlock-free (from every reachable state, some action is possible).”

Exercise 8.2

Prove the $\Rightarrow$ direction of the adequacy theorem in Lean: if $s \sim t$, then $s \models \varphi \iff t \models \varphi$ for all φ . Use induction on φ .

Part III

Types for Concurrency

Chapter 9

Session Types — Protocols as Types

“Propositions as sessions.”

Philip Wadler (2012)

9.1 Typing Communication

Kohei Honda (1956–2012)

Kohei Honda had a beautiful idea: if types can describe the structure of *data* (lists, trees, records), why not the structure of *communication*? A type like $!Int.?Bool.end$ describes a protocol: “send an integer, then receive a boolean, then done.” The dual type $?Int.!Bool.end$ describes the other side: “receive an integer, then send a boolean, then done.”

Honda’s 1993 paper introduced **session types** for the π -calculus. The key theorem: if both endpoints of a channel are typed with dual session types, then communication is *guaranteed* to be deadlock-free, type-safe, and eventually complete.

Honda (with Vasconcelos and Kubo, 1998) extended the theory to cover choice, recursion, and delegation (passing a session-typed channel to another process). Tragically, Honda died in 2012 at the age of 56, before seeing session types reach mainstream adoption. His work is now recognized as one of the most important contributions to programming language theory.

9.2 Session Type Syntax

Definition 9.1 — Session Types

$S ::= !T.S$	send a value of type T , continue as S
$?T.S$	receive a value of type T , continue as S
$S_1 \oplus S_2$	internal choice: sender picks a branch
$S_1 \& S_2$	external choice: receiver picks a branch
end	session complete
$\mu X.S$	recursive session (for protocols that loop)

Lean 4

```

-- Base data types for messages
inductive BaseType where
  | nat | bool | str
  deriving DecidableEq, Repr

-- Session types
inductive SessionType where
  | send    : BaseType → SessionType → SessionType -- !T.S
  | recv    : BaseType → SessionType → SessionType -- ?T.S
  | choose  : SessionType → SessionType → SessionType -- S1 ⊕ S2
  | offer   : SessionType → SessionType → SessionType -- S1 & S2
  | done    : SessionType -- end
  | var     : Nat → SessionType -- recursion variable
  | mu      : SessionType → SessionType -- μX.S
  deriving DecidableEq, Repr

```

9.3 Duality

The fundamental operation: every session type has a **dual** that describes the other endpoint of the channel.

Definition 9.2 — Session Type Duality

$$\begin{array}{ll}
 \overline{!T.S} = ?T.\overline{S} & \overline{?T.S} = !T.\overline{S} \\
 \overline{S_1 \oplus S_2} = \overline{S_1} \& \overline{S_2} & \overline{S_1 \& S_2} = \overline{S_1} \oplus \overline{S_2} \\
 \overline{\text{end}} = \text{end} & \overline{\mu X.S} = \mu X.\overline{S}
 \end{array}$$

Lean 4

```

def SessionType.dual : SessionType → SessionType
| .send t s    => .recv t s.dual
| .recv t s    => .send t s.dual
| .choose a b  => .offer a.dual b.dual
| .offer a b   => .choose a.dual b.dual
| .done       => .done
| .var n       => .var n
| .mu s       => .mu s.dual

-- Duality is an involution: dual (dual S) = S
theorem SessionType.dual_dual (S : SessionType) :
  S.dual.dual = S := by
  induction S with
  | send t s ih => simp [dual, ih]
  | recv t s ih => simp [dual, ih]
  | choose a b iha ihb => simp [dual, iha, ihb]
  | offer a b iha ihb => simp [dual, iha, ihb]
  | done => rfl
  | var n => rfl
  | mu s ih => simp [dual, ih]

```

9.4 Type Safety

The central guarantee: well-typed processes don't deadlock.

Theorem 9.1 — Session Type Safety

If process P has session type S on channel c , and process Q has session type $\bar{S}$ on c , then $(\nu c)(P \parallel Q)$ is:

- (i) **Type-safe:** messages always have the expected type.
- (ii) **Deadlock-free:** if both sides follow their types, communication always progresses.
- (iii) **Session-complete:** the protocol eventually reaches end on both sides.

Proof sketch. By **progress** and **preservation**:

- *Progress:* if the session hasn't ended, at least one side can take a step (because send is matched with receive in the dual).
- *Preservation:* after a communication step, the remaining session types are still dual (because duality “peels off” matching prefixes).

□

9.5 The Linear Logic Connection

Wadler's “Propositions as Sessions” (2012)

Philip Wadler discovered a remarkable correspondence: session types are propositions in **classical linear logic**, and well-typed processes are proofs. This is a Curry-Howard correspondence for concurrent computation:

Linear Logic		Session Types
$A \otimes B$ (tensor)	$\longleftrightarrow$	$!A.S$ (send A , continue S)
$A \wp B$ (par)	$\longleftrightarrow$	$?A.S$ (receive A , continue S)
$A \oplus B$ (plus)	$\longleftrightarrow$	$S_1 \oplus S_2$ (internal choice)
$A \& B$ (with)	$\longleftrightarrow$	$S_1 \& S_2$ (external choice)
$\mathbf{1}$ (unit)	$\longleftrightarrow$	end (session complete)
$A^\perp$ (negation)	$\longleftrightarrow$	$\overline{S}$ (duality)

The key is *linearity*: each channel is used exactly once (you send on it or receive on it, then move to the continuation). This is why each message is a resource consumed exactly once—the linear logic discipline prevents aliasing, duplication, and forgetting of channels. If you’ve read the lambda calculus book: the Curry-Howard correspondence says “proofs = programs” for sequential computation. Wadler’s correspondence says “proofs in linear logic = well-typed concurrent programs.”

Exercise 9.1

Define session types for a two-phase commit protocol: the coordinator sends a **prepare** message, each participant responds with **vote_commit** or **vote_abort**, then the coordinator sends the final **commit** or **abort** decision. Verify duality holds.

Exercise 9.2

Prove `SessionType.dual_dual` in Lean (the skeleton is above—fill in the details for each case).

Chapter 10

Practical Session Types for TypeScript

“In theory, there is no difference between theory and practice. In practice, there is.”

Attributed to Yogi Berra

10.1 What Session Types Would Look Like

Imagine if TypeScript could enforce protocol ordering:

TypeScript

```
// A session-typed channel
type WorkerProtocol =
  | { phase: 'send'; payload: Task; next: WorkerRecv }
  | { phase: 'done' };
type WorkerRecv =
  | { phase: 'recv'; payload: Result; next: WorkerDone };
type WorkerDone = { phase: 'done' };

// The type system would enforce:
// 1. You MUST send a Task before receiving a Result
// 2. You MUST receive a Result after sending a Task
// 3. After the exchange, the session is done
```

10.2 Current TypeScript Approximations

10.2.1 State Machine Pattern

TypeScript

```
// Encode protocol states as types
type Idle = { state: 'idle' };
type Sent = { state: 'sent'; taskId: string };
type Done = { state: 'done'; result: Result };

// Type-safe transitions
function sendTask(ch: Idle, task: Task): Sent { ... }
function recvResult(ch: Sent): Promise<Done> { ... }
// This won't compile: function recvResult(ch: Idle): ... -- type error!
```

This enforces ordering but has limitations: nothing prevents you from using a channel in state `Sent` twice (non-linearity), or from forgetting to call `recvResult` (no completion guarantee).

10.2.2 Discriminated Unions with XState

State machine libraries like XState provide runtime enforcement:

TypeScript

```
import { createMachine, interpret } from 'xstate';

const protocolMachine = createMachine({
  initial: 'idle',
  states: {
    idle: { on: { SEND_TASK: 'awaitingResult' } },
    awaitingResult: { on: { RECV_RESULT: 'done' } },
    done: { type: 'final' },
  },
});
```

10.3 What's Missing in TypeScript

For full session type safety, TypeScript would need:

1. **Linear types**: prevent a channel from being used twice or forgotten. Rust's ownership system provides this; TypeScript doesn't.
2. **Higher-kinded types**: abstract over protocol structure (e.g., “for any protocol S , ...”). This connects back to the lambda calculus book's discussion of System $F\omega$.
3. **Dependent types**: compute the next protocol state from message values (e.g., branching on the content of a received message).

10.4 Languages with Real Session Type Support

- **Rust**: ownership = affine types, so session type libraries (`session-types` crate) work well. A consumed channel can't be reused.
- **Haskell**: the `simple-sessions` library encodes session types using indexed monads.
- **Scribble/Links**: research languages designed around multiparty session types.

Exercise 10.1

Implement a TypeScript wrapper around `Worker` threads that uses the state machine pattern to enforce a request-response protocol. The API should make it a type error to receive before sending or to send twice.

Chapter 11

Multiparty Session Types

“The whole is more than the sum of its parts.”

Aristotle

11.1 Beyond Two Parties

Honda, Yoshida, and Carbone (2008)

Honda, Nobuko Yoshida, and Marco Carbone extended session types from two-party to **multiparty** communication. A *global type* describes the entire protocol from a bird’s-eye view; each participant’s *local type* is obtained by *projecting* the global type onto that participant.

The key theorem: if each participant follows their local type, the global protocol is satisfied. This means you can specify a complex multi-service interaction once (the global type) and automatically derive each service’s obligations.

The Scribble protocol language (Imperial College London), based on multiparty session types, has been used by Red Hat, VMware, and the Ocean Observatories Initiative.

11.2 Global Types

Definition 11.1 — Global Type

A **global type** specifies the complete interaction:

$$G ::= p \rightarrow q : T . G \mid p \rightarrow q : \{l_i : G_i\}_{i \in I} \mid \text{end} \mid \mu X . G$$

- $p \rightarrow q : T . G$ — participant p sends a value of type T to participant q , then continue as G .
- $p \rightarrow q : \{l_i : G_i\}$ — p sends one of several labels l_i to q , selecting the corresponding continuation G_i .

11.2.1 Example: Client-Server-Database

```

G = Client → Server : Query.
  Server → Database : SQL.
  Database → Server : Rows.
  Server → Client : Response.
end

```

Lean 4

```

-- Participants
inductive Role where
  | client | server | database
  deriving DecidableEq, Repr

-- Global types
inductive GlobalType where
  | msg      : Role → Role → BaseType → GlobalType → GlobalType
  | choice   : Role → Role → List (String × GlobalType) → GlobalType
  | done     : GlobalType
  | mu       : GlobalType → GlobalType
  | var      : Nat → GlobalType
  deriving Repr

```

11.3 Projection

Each participant extracts their local view of the protocol:

Definition 11.2 — Projection

The **projection** of global type G onto participant r is:

$$(p \rightarrow q : T.G) \upharpoonright r = \begin{cases} !T.(G \upharpoonright r) & \text{if } r = p \\ ?T.(G \upharpoonright r) & \text{if } r = q \\ G \upharpoonright r & \text{if } r \neq p, q \end{cases}$$

Theorem 11.1 — Multiparty Session Safety

If global type G is well-formed (no ambiguity in projections), and each participant r follows their local type $G \upharpoonright r$, then:

- (i) Communication is deadlock-free.
- (ii) Messages are type-safe.
- (iii) The protocol completes.

11.4 Connection to Choreographic Programming

Intuition

Multiparty session types enable **choreographic programming**: write the global protocol once, then *automatically generate* each participant’s code. This is the concurrent analogue of “write the specification, derive the implementation”—a theme that connects to formal verification throughout computer science.

For TypeScript: imagine writing the global protocol for your microservice architecture and having the TypeScript types for each service’s API automatically derived. gRPC protocol definitions, GraphQL subscriptions, and WebSocket protocols are all steps toward this vision.

Exercise 11.1

Define a global type for an authentication handshake: Client sends credentials to AuthServer, AuthServer sends a token (or rejection) to Client, on success Client sends the token to ResourceServer, ResourceServer sends data to Client. Project onto each of the three roles.

Part IV

Capstone

Chapter 12

Capstone — Verified Worker Pool Protocol

“Move fast and break things”
becomes
“Move fast and *prove* things.”

The promise of verified concurrency

12.1 The Protocol

We design, formalize, and verify a worker pool protocol for Node.js:

1. A **Dispatcher** maintains a queue of tasks and a pool of workers.
2. **Workers** report availability, receive tasks, and return results.
3. Workers can **join and leave** dynamically (π -calculus, not just CCS).

12.1.1 Global Type

$$\begin{aligned} G_{\text{pool}} &= \mu X. \text{Worker} \rightarrow \text{Dispatcher} : \{\text{ready} : G_{\text{task}}, \text{leave} : \text{end}\} \\ G_{\text{task}} &= \text{Dispatcher} \rightarrow \text{Worker} : \text{Task}. \\ &\quad \text{Worker} \rightarrow \text{Dispatcher} : \text{Result}. \\ &\quad X \end{aligned}$$

A worker announces it’s ready (or leaves). If ready, the dispatcher sends a task, the worker returns a result, and the protocol loops.

12.1.2 π -Calculus Formalization

Lean 4

```

-- The dispatcher process
def dispatcher (tasks : List Name) (pool : Name) : Pi :=
  -- Listen for worker registrations on pool channel
  .recv pool "worker" <|
  -- Send a task to the worker
  match tasks with
  | [] => .nil -- no more tasks
  | t :: ts =>
    .send "worker" t <|
    -- Wait for result
    .recv "worker" "result" <|
    dispatcher ts pool

-- A worker process
def worker (pool : Name) : Pi :=
  -- Register with dispatcher
  .send pool "self" <| -- scope extrusion: pass our channel
  -- Receive task
  .recv "self" "task" <|
  -- Process and return result (modeled as tau + send)
  Pi.res "result" <|
  .send "self" "result" <|
  worker pool -- loop: re-register

```

12.2 Properties to Prove

Theorem 12.1 — Worker Pool Deadlock Freedom

If there is at least one worker and the dispatcher has tasks, the system always makes progress: there is always a possible transition.

Proof sketch. 1. If a worker is in the “ready” state and the dispatcher has a task, they can communicate (the ready/task exchange).
 2. After receiving a task, the worker eventually produces a result (by the structure of the worker process).
 3. After receiving the result, the dispatcher loops and the worker re-registers.
 4. At every state, at least one communication can occur. □

Theorem 12.2 — No Message Loss

Every task sent by the dispatcher is eventually paired with a result received from a worker.

Theorem 12.3 — Session Type Safety

Workers always receive well-formed tasks (matching the session type), and the dispatcher always receives well-formed results.

12.3 The TypeScript Implementation

TypeScript

```

import { Worker, MessageChannel, MessagePort } from 'worker_threads';

interface Task { id: string; data: unknown; }
interface Result { id: string; output: unknown; }

class VerifiedWorkerPool {
  private workers: Map<string, MessagePort> = new Map();
  private taskQueue: Task[] = [];
  private idle: string[] = [];

  constructor(private workerScript: string, private size: number) {
    for (let i = 0; i < size; i++) this.spawnWorker(`w${i}`);
  }

  private spawnWorker(id: string) {
    const { port1, port2 } = new MessageChannel(); // (v port1, port2)
    const w = new Worker(this.workerScript);
    w.postMessage({ port: port2 }, [port2]); // scope extrusion!

    this.workers.set(id, port1);

    port1.on('message', (msg: { type: string } & Record<string, unknown>) => {
      if (msg.type === 'ready') { // Worker → Dispatcher:
        ↪ ready
        this.idle.push(id);
        this.dispatch();
      } else if (msg.type === 'result') { // Worker → Dispatcher:
        ↪ result
        this.handleResult(msg as unknown as Result);
      }
    });
  }

  private dispatch() {
    while (this.idle.length > 0 && this.taskQueue.length > 0) {
      const wid = this.idle.shift()!;
      const task = this.taskQueue.shift()!;
      const port = this.workers.get(wid)!;
      port.postMessage({ type: 'task', ...task }); // Dispatcher → Worker:
      ↪ task
    }
  }

  submit(task: Task) {
    this.taskQueue.push(task);
    this.dispatch();
  }

  private handleResult(result: Result) {
    // Process result...
  }
}

```

Exercise 12.1

Extend the protocol to handle worker crashes: if a worker fails to return a result within a timeout, the dispatcher reassigns the task to another worker. Formalize the extended protocol and prove that every task eventually completes (assuming at least one worker is always available).

Exercise 12.2

Prove in Lean that the worker pool protocol satisfies the deadlock-freedom property. *Hint:* show that in every reachable state, at least one communication rule (from the π -calculus operational semantics) is applicable.

Chapter 13

Looking Back — The Science of Concurrency

13.1 The Intellectual Arc

13.2 Where the Models Live in Industry

13.3 What We Didn't Cover

-

-
-
-
-

13.4 Cross-Book Connections

-
-
-

“The purpose of computing is insight, not numbers.”

— *Richard Hamming*

*The purpose of concurrency theory is
correctness, not performance.
(Performance is a bonus.)*

Appendix A: CCS and π -Calculus Quick Reference

CCS Syntax

π -Calculus Extensions

Session Type Syntax

Appendix B: Lean 4 Definitions Index

Appendix C: Historical Timeline